

Taller Práctico

Desarrollo de una Base de Datos

Euris Rodríguez 8-1013-2315

GITHUB: [GITHUB LINK](#)

1. Objetivo del Taller

El objetivo del taller es desarrollar un prototipo funcional de sistema backend que incluya:

- Diseño de base de datos
- Implementación de la base de datos en MariaDB
- Uso de Docker para ejecutar el servicio de base de datos
- Desarrollo de una API REST con Python y Flask
- Conexión entre la API y la base de datos mediante conectores
- Publicación del proyecto en un repositorio

El sistema desarrollado es un Sistema de Gestión de Expedientes.

2. Tecnologías utilizadas

Tecnología	Uso
Docker	Contenerización de la base de datos
MariaDB	Motor de base de datos
Python	Lenguaje backend
Flask	Framework para API REST
mysql-connector-python	Conector entre Python y MariaDB

3. Estructura actual del proyecto

TALLER/

|

└─ api/

|

└─ app.py

|

└─ database.py

|

└─ requirements.txt

|

└─ docker-compose.yml

4. Instalación y uso de Docker

Primero se verificó que Docker estuviera instalado.

Verificar Docker

`docker --version`

Resultado:

```
PS C:\Users\jw818\Desktop\taller> docker --version
Docker version 29.2.1, build a5c7197
❖ PS C:\Users\jw818\Desktop\taller>
```

Ver contenedores activos

`docker ps`

Esto muestra los contenedores actualmente ejecutándose.

```
PS C:\Users\jw818\Desktop\taller> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
PS C:\Users\jw818\Desktop\taller>
```

5. Creación del contenedor de MariaDB

Se creó un contenedor que ejecuta MariaDB y expone el puerto al sistema.

Comando utilizado

```
docker run -d --name expedientes_db -e MARIADB_ROOT_PASSWORD=root123 -e MARIADB_DATABASE=sistema_expedientes -p 3307:3306 mariadb:11
```

Explicación del comando

Parámetro	Función
docker run	Crear contenedor
-d	Ejecutar en segundo plano
--name expedientes_db	Nombre del contenedor
-e MARIADB_ROOT_PASSWORD	Contraseña root
-e MARIADB_DATABASE	Base creada automáticamente
-p 3307:3306	Puerto local → puerto del contenedor
mariadb:11	Imagen de MariaDB

6. Verificación del contenedor

Para verificar que el contenedor esté ejecutándose:

```
docker ps
```

Resultado:

```
PS C:\Users\jw818\Desktop\taller> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
d19f72a64e67   mariadb:11     "docker-entrypoint.s..." 35 hours ago   Up 9 seconds  0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp  expedientes_db
PS C:\Users\jw818\Desktop\taller>
```

7. Acceso a la base de datos dentro del contenedor

Para entrar al cliente de MariaDB dentro del contenedor:

```
docker exec -it expedientes_db mariadb -u root -p
```

Contraseña: root123

```
PS C:\Users\jw818\Desktop\taller> docker exec -it expedientes_db mariadb -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 11.8.6-MariaDB-ubu2404 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
```

8. Verificación de bases de datos

Una vez dentro de MariaDB se verificaron las bases de datos disponibles.

Comando utilizado

```
SHOW DATABASES;
```

Función del comando

Permite listar todas las bases de datos existentes dentro del servidor MariaDB.

```
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database                |
+-----+
| information_schema      |
| mysql                   |
| performance_schema     |
| sistema_expedientes    |
| sys                     |
+-----+
5 rows in set (0.005 sec)

MariaDB [(none)]>
```

9. Selección de la base de datos

Para trabajar con la base de datos creada se utilizó el siguiente comando.

Comando utilizado

USE sistema_expedientes;

Función del comando

Permite seleccionar la base de datos que se utilizará para ejecutar las consultas SQL.

```
MariaDB [(none)]> USE sistema_expedientes;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A  
  
Database changed  
MariaDB [sistema_expedientes]> |
```

10. Creación de las tablas del sistema

Tabla usuarios

Comando utilizado

```
CREATE TABLE usuarios (  
id_usuario INT AUTO_INCREMENT PRIMARY KEY,  
nombre VARCHAR(100),  
usuario VARCHAR(50),  
password VARCHAR(100),  
rol VARCHAR(50)  
);
```

Descripción

Esta tabla almacena la información de los usuarios del sistema.

```
MariaDB [sistema_expedientes]> DESCRIBE usuarios;  
+-----+-----+-----+-----+-----+-----+  
| Field      | Type          | Null | Key | Default | Extra          |  
+-----+-----+-----+-----+-----+-----+  
| id_usuario | int(11)       | NO   | PRI | NULL    | auto_increment |  
| nombre     | varchar(100)  | YES  |     | NULL    |                |  
| usuario    | varchar(50)   | YES  |     | NULL    |                |  
| password   | varchar(100)  | YES  |     | NULL    |                |  
| rol        | varchar(50)   | YES  |     | NULL    |                |  
+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.002 sec)  
  
MariaDB [sistema_expedientes]> 
```

Tabla clientes

```
CREATE TABLE clientes (  
id_cliente INT AUTO_INCREMENT PRIMARY KEY,  
nombre VARCHAR(100),  
correo VARCHAR(100),  
telefono VARCHAR(20)  
);
```

```
MariaDB [sistema_expedientes]> DESCRIBE clientes;  
+-----+-----+-----+-----+-----+-----+  
| Field      | Type          | Null | Key | Default | Extra          |  
+-----+-----+-----+-----+-----+-----+  
| id_cliente | int(11)       | NO   | PRI | NULL    | auto_increment |  
| nombre     | varchar(100)  | YES  |     | NULL    |                |  
| correo     | varchar(100)  | YES  |     | NULL    |                |  
| telefono   | varchar(20)   | YES  |     | NULL    |                |  
+-----+-----+-----+-----+-----+-----+  
4 rows in set (0.002 sec)  
  
MariaDB [sistema_expedientes]> 
```

Tabla expedientes

```
CREATE TABLE expedientes (  
id_expediente INT AUTO_INCREMENT PRIMARY KEY,  
titulo VARCHAR(150),  
descripcion TEXT,  
fecha_creacion DATE,  
id_cliente INT,  
FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)  
);
```

```
MariaDB [sistema_expedientes]> DESCRIBE expedientes;  
+-----+-----+-----+-----+-----+-----+  
| Field          | Type          | Null | Key | Default | Extra          |  
+-----+-----+-----+-----+-----+-----+  
| id_expediente | int(11)       | NO   | PRI | NULL    | auto_increment |  
| titulo        | varchar(150)  | YES  |     | NULL    |                |  
| descripcion   | text          | YES  |     | NULL    |                |  
| fecha_creacion | date          | YES  |     | NULL    |                |  
| id_cliente    | int(11)       | YES  | MUL | NULL    |                |  
+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.002 sec)  
  
MariaDB [sistema_expedientes]> 
```

Tabla documentos

```
CREATE TABLE documentos (  
id_documento INT AUTO_INCREMENT PRIMARY KEY,  
nombre_documento VARCHAR(150),  
tipo VARCHAR(50),  
fecha_subida DATE,  
id_expediente INT,  
FOREIGN KEY (id_expediente) REFERENCES expedientes(id_expediente)  
);
```

```
MariaDB [sistema_expedientes]> DESCRIBE documentos;
```

Field	Type	Null	Key	Default	Extra
id_documento	int(11)	NO	PRI	NULL	auto_increment
nombre_documento	varchar(150)	YES		NULL	
tipo	varchar(50)	YES		NULL	
fecha_subida	date	YES		NULL	
id_expediente	int(11)	YES	MUL	NULL	

```
5 rows in set (0.009 sec)
```

```
MariaDB [sistema_expedientes]> 
```


11. Verificación de tablas creadas

Para verificar que todas las tablas se crearon correctamente se ejecutó el siguiente comando.

SHOW TABLES;

Función

Permite listar todas las tablas dentro de la base de datos seleccionada.

```
MariaDB [sistema_expedientes]> SHOW TABLES;
+-----+
| Tables_in_sistema_expedientes |
+-----+
| clientes                       |
| documentos                     |
| expedientes                    |
| usuarios                      |
+-----+
4 rows in set (0.002 sec)

MariaDB [sistema_expedientes]> 
```

12. Desarrollo de la API REST

Posteriormente se desarrolló una API REST utilizando Python y el framework Flask.

Instalación de dependencias

Se instalaron las librerías necesarias para el proyecto.

```
pip install flask
```

```
pip install mysql-connector-python
```

Función

- Flask permite crear el servidor web y los endpoints de la API.
- mysql-connector-python permite conectar Python con la base de datos MariaDB.

13. Conexión a la base de datos desde Python

Se creó un archivo llamado database.py para gestionar la conexión a la base de datos.

Código utilizado:

```
import mysql.connector

def get_connection():
    connection = mysql.connector.connect(
        host="localhost",
        port=3307,
        user="root",
        password="root123",
        database="sistema_expedientes"
    )
    return connection
```

Descripción

Este archivo establece la conexión entre la aplicación Python y la base de datos.

14. Creación de la API con Flask

Se creó el archivo app.py que contiene la lógica de la API.

```
from flask import Flask, jsonify
from database import get_connection

app = Flask(__name__)

@app.route("/")
def home():
    return {"mensaje": "API del sistema de expedientes funcionando"}

@app.route("/usuarios", methods=["GET"])
def obtener_usuarios():
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)

    cursor.execute("SELECT * FROM usuarios")
    usuarios = cursor.fetchall()

    cursor.close()
    conn.close()

    return jsonify(usuarios)

if __name__ == "__main__":
    app.run(debug=True)
```

15. Ejecución de la API

Para ejecutar la aplicación se utilizó el siguiente comando.

```
cd api
```

```
python app.py
```

Función

- `cd api` entra a la carpeta donde se encuentra la aplicación.
- `python app.py` ejecuta el servidor Flask.

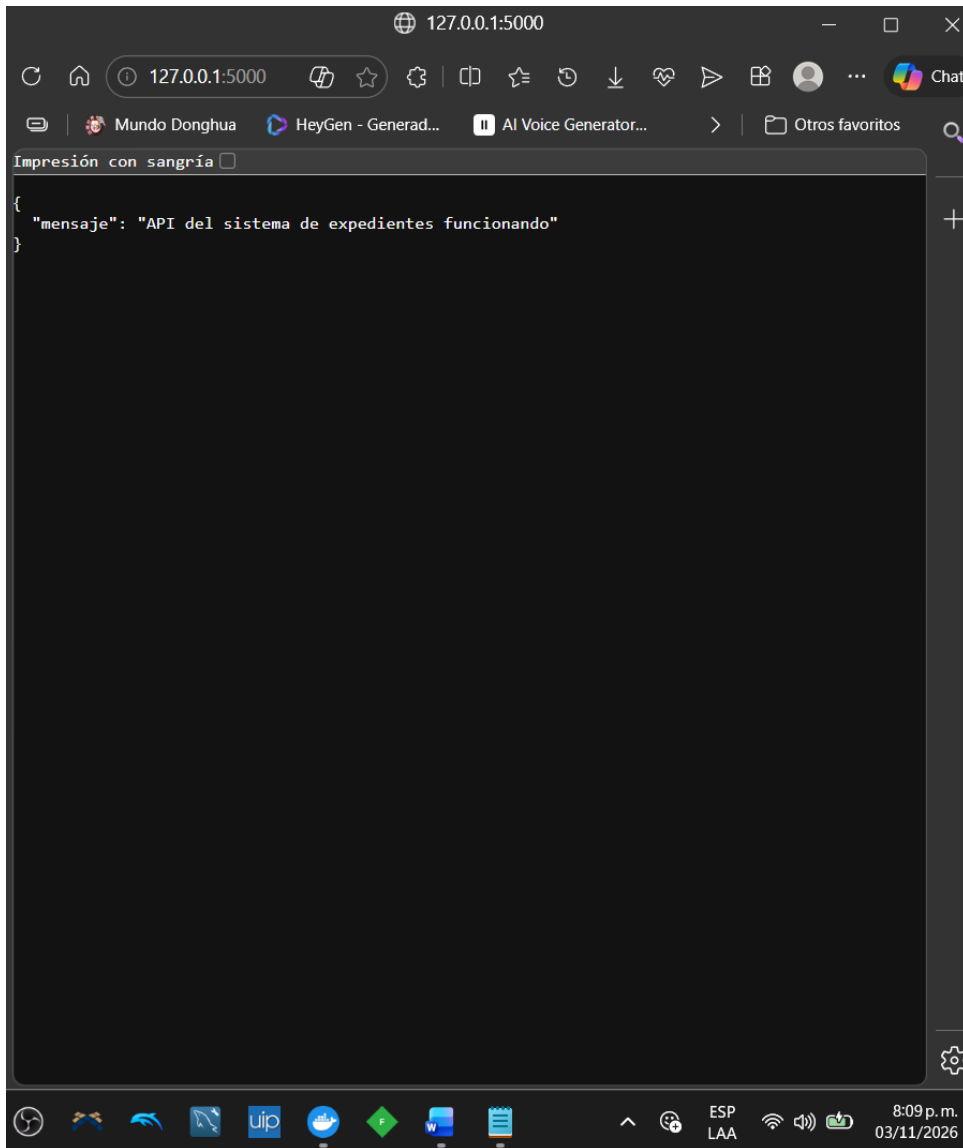
```
● PS C:\Users\jw818\Desktop\taller> cd api
○ PS C:\Users\jw818\Desktop\taller\api> python app.py
  * Serving Flask app 'app'
  * Debug mode: on
WARNING: This is a development server. Do not use it in a production
deployment. Use a production WSGI server instead.
  * Running on http://127.0.0.1:5000
Press CTRL+C to quit
  * Restarting with stat
  * Debugger is active!
  * Debugger PIN: 187-191-675
```

16. Prueba de la API REST

Una vez ejecutada la aplicación se realizaron pruebas utilizando el navegador.

Endpoint probado:

http://127.0.0.1:5000



17. Conclusiones

El desarrollo del taller me permitió comprender el proceso completo de creación de una aplicación backend que interactúa con una base de datos.

Se logró diseñar e implementar una base de datos utilizando MariaDB dentro de un contenedor Docker, lo cual facilita la portabilidad del sistema y su despliegue en diferentes entornos.

Además, se desarrolló una API REST utilizando Flask que permitirá consultar la información almacenada en la base de datos, demostrando la integración entre el backend y el sistema de almacenamiento de datos.

El uso de tecnologías como Docker y Flask representa una práctica común en el desarrollo moderno de aplicaciones.

18. Lecciones aprendidas

Durante el desarrollo del taller se adquirieron varios conocimientos importantes:

- Uso de contenedores Docker para ejecutar servicios de base de datos.
- Creación y administración de bases de datos utilizando MariaDB.
- Diseño de tablas y relaciones entre entidades.
- Conexión entre aplicaciones Python y bases de datos mediante conectores.
- Desarrollo de APIs REST utilizando Flask.
- Uso de endpoints para consultar información en formato JSON.

Estas habilidades son fundamentales para el desarrollo de aplicaciones backend modernas.

Diagrama

